0.70. Plus fluide : la lettre au pixel près

sprite(0, px, py, ALPHABET[0]) : la lettre devient un lutin, qui avance d'UN pixel par image au lieu de sauter d'une case. Le mouvement devient fluide.

1. Ce qu'il faut comprendre

Pourquoi la lettre saute : jusqu'ici, elle était **posée sur la grille** avec `poser`. Une case fait **8 × 8 pixels** : chaque pas la faisait donc sauter de 8 pixels d'un coup, environ 7 fois par seconde. L'œil voit des sauts.

Ce qui est nouveau ici : un lutin, avec `sprite(numero, x, y, tuile)`. Un lutin n'est **pas** sur la grille : il flotte au-dessus du décor, et se place **au pixel près**. On le fait avancer d'**un seul pixel par image**, 60 fois par seconde : le mouvement devient **fluide**.

• `numero` : le numéro du lutin, de 0 à 39 (la console en a 40). Ici, `0`, le premier.

• `px`, `py` : sa place **en pixels**, et plus en cases. L'écran fait **160 pixels** de large et **144** de haut. Le milieu, pour une lettre de 8 pixels : (160 − 8) ÷ 2 = **76**, et (144 − 8) ÷ 2 = **68**.

• `tuile` : ce qu'il montre, `ALPHABET[0]`, le A.

Plus besoin d'`effacer` ni d'`attente` : un lutin se **déplace**, il ne se réécrit pas. On lui donne sa nouvelle place à chaque image, et la console le dessine là, sans rien laisser derrière. Et comme un pixel est petit, on peut avancer à chaque image, sans attendre.

Les bords : `px` va de 0 à **152** (160 − 8, pour que la lettre reste entière), `py` de 0 à **136** (144 − 8).

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <bouton>` → 0.65.5. La fonction bouton() — lire la manette
> - `#include <sprite>` → 0.69.4. La fonction sprite() — un lutin au pixel près
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **bouton()** — Une fonction de la console qui dit si un bouton de la manette est enfoncé : 1 oui, 0 non. Les boutons : A, B, HAUT, BAS, GAUCHE, DROITE, START, SELECT.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **lutin (sprite)** — Un dessin qui bouge au pixel près, par-dessus le décor : le héros, un ennemi. sprite() en place un de 8 × 8, sprite16() un de 16 × 16. La console en a 40.
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **opérateurs logiques** — && « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
- **pixel** — Un tout petit carré de couleur ; l'écran en a 160 de large et 144 de haut.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// px, py : voir l'onglet variables.h`	Un commentaire : une phrase pour celui qui lit (« px, py : voir l'onglet variables.h »). La console l'ignore.
`#include <bouton>     // lit un bouton de la manette`	Demande à la console « bouton » (ce que c'est : lit un bouton de la manette). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « lit un bouton de la manette ».)
`#include <sprite>     // place un lutin de 8 × 8 au pixel près`	Demande à la console « sprite » (ce que c'est : place un lutin de 8 × 8 au pixel près). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « place un lutin de 8 × 8 au pixel près ».)
`#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include "variables.h"`	Verse ici le fichier « variables.h » (un autre onglet du programme) : tout se passe comme si son texte était écrit à cet endroit.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();                                  // 60 fois par seconde`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « 60 fois par seconde ».)
`// UN pixel par image, tant que la flèche est tenue : 60 pixels par`	Un commentaire : une phrase pour celui qui lit (« UN pixel par image, tant que la flèche est tenue : 60 pixels par »). La console l'ignore.
`// seconde, sans à-coups. (Avec poser, c'était 8 pixels d'un coup.)`	Un commentaire : une phrase pour celui qui lit (« seconde, sans à-coups. (Avec poser, c'était 8 pixels d'un coup.) »). La console l'ignore.
`if (bouton(DROITE) && px < 152) px++;     // 152 = 160 - 8 : le bord droit`	Une décision : si le bouton DROITE est enfoncé et px est plus petit que 152 alors on fait « px++ ». Sinon, on passe. (Le commentaire le dit : « 152 = 160 - 8 : le bord droit ».)
`if (bouton(GAUCHE) && px > 0)   px--;`	Une décision : si le bouton GAUCHE est enfoncé et px est plus grand que 0 alors on fait « px-- ». Sinon, on passe.
`if (bouton(BAS) && py < 136)    py++;     // 136 = 144 - 8 : le bord du bas`	Une décision : si le bouton BAS est enfoncé et py est plus petit que 136 alors on fait « py++ ». Sinon, on passe. (Le commentaire le dit : « 136 = 144 - 8 : le bord du bas ».)
`if (bouton(HAUT) && py > 0)     py--;`	Une décision : si le bouton HAUT est enfoncé et py est plus grand que 0 alors on fait « py-- ». Sinon, on passe.
`// Le lutin, morceau par morceau :`	Un commentaire : une phrase pour celui qui lit (« Le lutin, morceau par morceau : »). La console l'ignore.
`//   sprite(0, px, py, ALPHABET[0]);`	Un commentaire : une phrase pour celui qui lit (« sprite(0, px, py, ALPHABET[0]); »). La console l'ignore.
`//          |  |   |   |`	Un commentaire : une phrase pour celui qui lit (« |||| »). La console l'ignore.
`//          |  |   |   +-- tuile  : ce qu'il montre, la lettre A`	Un commentaire : une phrase pour celui qui lit (« ||| +-- tuile : ce qu'il montre, la lettre A »). La console l'ignore.

`//          |  +---+------ px, py : sa place EN PIXELS (et non en cases)`	Un commentaire : une phrase pour celui qui lit (« | +---+------ px, py : sa place EN PIXELS (et non en cases) »). La console l'ignore.
`//          +------------- numero : le lutin n° 0 (la console en a 40, de 0 à 39)`	Un commentaire : une phrase pour celui qui lit (« +------------- numero : le lutin n° 0 (la console en a 40, de 0 à 39) »). La console l'ignore.
`// Pas d'effacer : un lutin se DÉPLACE, il ne laisse rien derrière lui.`	Un commentaire : une phrase pour celui qui lit (« Pas d'effacer : un lutin se DÉPLACE, il ne laisse rien derrière lui. »). La console l'ignore.
`sprite(0, px, py, ALPHABET[0]);`	Appel de la fonction sprite de la console, avec les arguments 0, px, py, ALPHABET[0]. Ce qu'elle fait : un lutin 8 × 8 au pixel près. Le 5e argument le retourne : MIROIR_X, MIROIR_Y, DERRIERE. Un NOMBRE de 2 ou plus en 5e argument est une TAILLE : le dessin agrandi d'autant (le dessin n'est écrit qu'une fois). sprite(0, 36, 60, ROND, 4).
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **340 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage            55 o  ← la console, à l'allumage  — la mise en route de la console
  main()              113 o  ← démarrage  — ta fonction
  VBlank                8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage        28 o  ← image()  — attend l'image suivante
  AttendreVBlank       16 o  ← démarrage  — attend que l'écran soit libre
  RangerLutins         11 o  ← démarrage  — range les 40 lutins hors de l'écran
  InstallerTransfert   15 o  ← démarrage  — installe la copie des lutins en mémoire rapide
  CopierTuiles         31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  LireManette          37 o  ← bouton()  — lit les huit boutons
  Tuiles               16 o  ← CopierTuiles  — 1 lettre sur 43
  DonneesTransfert     10 o  ← InstallerTransfert  — la petite routine qui copie les lutins
Lettres gravées : A
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond,
EffacerPanneau, Notes
Fonctions du langage appelées :
  sprite() × 1    23 o sur place, dans main()
  bouton() × 4    20 o sur place, dans main()  + LireManette (37 o)
  image()  × 1     6 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// px, py : voir l'onglet variables.h

#include <bouton>     // lit un bouton de la manette
#include <sprite>     // place un lutin de 8 × 8 au pixel près
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();                                  // 60 fois par seconde

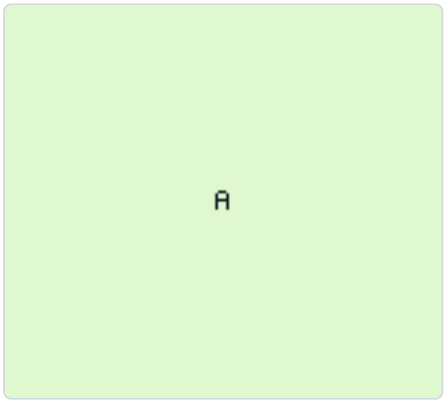
    // UN pixel par image, tant que la flèche est tenue : 60 pixels par
    // seconde, sans à-coups. (Avec poser, c'était 8 pixels d'un coup.)
    if (bouton(DROITE) && px < 152) px++;     // 152 = 160 - 8 : le bord droit
    if (bouton(GAUCHE) && px > 0)   px--;
    if (bouton(BAS) && py < 136)    py++;     // 136 = 144 - 8 : le bord du bas
    if (bouton(HAUT) && py > 0)     py--;

    // Le lutin, morceau par morceau :
    //
    //   sprite(0, px, py, ALPHABET[0]);
    //          |  |   |   |
    //          |  |   |   +-- tuile  : ce qu'il montre, la lettre A
    //          |  +---+------ px, py : sa place EN PIXELS (et non en cases)
    //          +-------------- numero : le lutin n° 0 (la console en a 40, de 0 à 39)
    //
    // Pas d'effacer : un lutin se DÉPLACE, il ne laisse rien derrière lui.
    sprite(0, px, py, ALPHABET[0]);
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

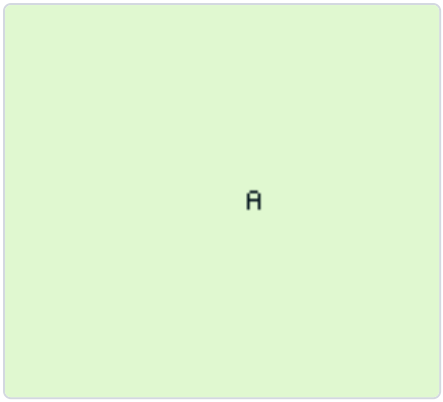
la toute première image — la console vient de démarrer — image 1



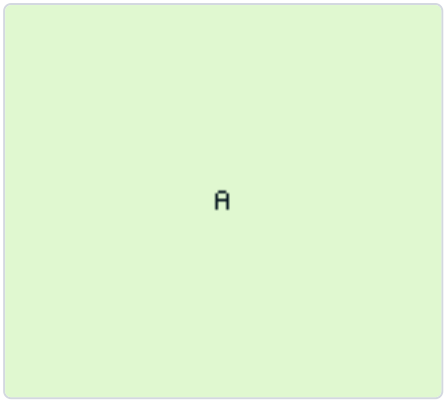
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



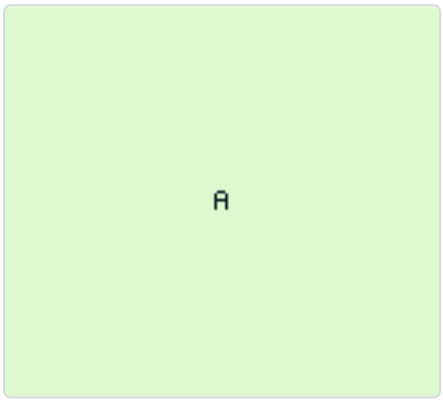
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

> Un A au milieu de l'écran, qui glisse doucement avec les flèches, pixel par pixel, sans sauter de case en case.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le lutin A part du milieu (76, 68) (76, 68)
- ✓ 30 images à droite : 30 pixels, un par image (30)
- ✓ 20 images en bas : 20 pixels (20)
- ✓ il s'arrête au bord gauche (0) (x = 0)

Ensuite : **0.70.1. Plus fluide : la lettre au pixel près — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++